

DiGamma: Domain-aware Genetic Algorithm for HW-Mapping Co-optimization for DNN Accelerators

Sheng-Chun Kao*, Michael Pellauer†, Angshuman Parashar†, Tushar Krishna*

* Georgia Institute of Technology † NVIDIA

* ska06@gatech.edu, tushar@ece.gatech.edu † {mpellauer, aparashar}@nvidia.com

Abstract—The design of DNN accelerators includes two key parts: HW resource configuration and mapping strategy. Intensive research has been conducted to optimize each of them independently. Unfortunately, optimizing for both together is extremely challenging due to the extremely large cross-coupled search space. To address this, in this paper, we propose a HW-Mapping co-optimization framework, an efficient encoding of the immense design space constructed by HW and Mapping, and a domain-aware genetic algorithm, named DiGamma, with specialized operators for improving search efficiency. We evaluate DiGamma with seven popular DNNs models with different properties. Our evaluations show DiGamma can achieve (geomean) 3.0x and 10.0x speedup, comparing to the best-performing baseline optimization algorithms, in edge and cloud settings.

I. INTRODUCTION

Specialized DNN accelerators design has become a hot topic and kept breaking through state-of-the-art performance for both training and inference [1], [2], [8]. The efficiency of a DNN accelerator is decided by its hardware (HW) resource configuration and the applied DNN mapping (dataflow + tiling) strategy, where both have been shown independently to be able to impact the accelerator performance by several orders [3]–[5].

The design space for HW configuration and mappings is extremely large. For a given accelerator with fixed HW resources, the number of ways to map the DNN computation on it can be as large as $O(10^{24})$ [4]. Meanwhile, for a given mapping strategy and fixed chip area budget, there can be $O(10^{12})$ possible HW implementations [3]. We elaborate on these in Sec. II-C. Many accelerators today either heavily rely on (i) heuristic and expert knowledge to strike the balance of resource allocation between compute resource, memory, and mapping [1], [2], [8] (manual-tuned *Fixed HW-Mapping*), or (ii) using AI/ML methods for finding efficient mappings given fixed hardware at compile-time [4] (*Mapping-opt*), or (iii) sizing the hardware resources at design-time assuming a fixed mapping strategy [3] (*HW-opt*). We summarize the related works in Fig. 1. One main challenge of these schemes is that they still include full or part of human-in-the-loop manual-tuning process for deciding a fixed mapping, a fixed HW, or both, which stacks up engineering cost. On the flip-side, without putting in this costly repeated manual-tuning effort, DNN accelerators may end up sub-optimal performance once the workload (the target DNN model) changes.

With the growth of AutoML, optimizing HW and Mapping together automatically with AI/ML offers a potential solution. However, how to effectively co-optimize the HW and mapping is still an open question. To address this, we propose a HW-Mapping co-optimization framework (Co-opt Framework) and

Work	HW Space		Mapping Space				Co-opt	Search Technique
	PE Array Sizing	Buffer Sizing	Tile	Order	Parallelism	Clustering		
RELEASE (RL4RealLife'19)	x	x	✓	x	x	x	-	Deep RL
ConfuciusX (MICRO'20)	✓	✓	x	x	x	x	-	Deep RL
Interstellar (ASPLOS'20)	✓	✓	x	x	✓	x	x	Two-step Heuristic
AutoSA (FPGA'21)	✓	x	x	x	x	x	-	Exhaustive Search
AutoTVM (OSDI'18)	x	x	✓	✓	✓	✓	-	Simulated Anneal
Timeloop (ISPASS'19)	x	x	✓	✓	✓	✓	-	Pruned Search
dMazeRunner (TECS'19)	x	x	✓	✓	✓	✓	-	Pruned Search
SIMBA (MICRO'19)	x	x	✓	✓	✓	✓	-	Random Search
GAMMA (ICCAD'20)	x	x	✓	✓	✓	✓	-	GA
FlexTensor (ASPLOS'20)	x	x	✓	✓	✓	✓	-	RL
CoSA (ISCA'21)	x	x	✓	✓	✓	✓	-	Mixed-Integer Programming
MindMapping (ASPLOS'21)	x	x	✓	✓	✓	✓	-	Gradient-based
HASCO (ISCA'21)	x	✓	✓	✓	✓	✓	x	Two-step Opt: (HW) Bayesian Opt + (Map) RL
DiGamma	✓	✓	✓	✓	✓	✓	✓	Co-opt with Specialized GA

Two-step opt/heuristic: Two steps (two search space) are optimized sequentially and independently.
Tiling: Tile size, tensor splitting. **Order:** Compute order. **Parallelism:** Spatial Map. **Clustering:** PE array reshaping, buffer hierarchy management, choice for levels of parallelism.

Fig. 1: State-of-the-Art HW and Mapping optimization frameworks.

an optimization algorithm, named DiGamma ($\mathcal{F}$), which search the HW resources configuration and the optimized mapping simultaneously. We make the following contributions:

- We propose a HW-Mapping co-optimization framework (Co-opt Framework), which takes in any DNN model(s), design objective, budget, and constraint, and generates an accelerator design point, HW (i.e., numbers of PEs, number of memory levels, sizes of buffers at each level) and mapping (i.e., parallelism, loop order, clustering, tile sizes). We abstract the detail of performance modeling for different DNNs, chip constraints and provide a generic interface, where many existing optimization algorithms can be plugged in, as shown via our experiments in Sec. V-B.
- We propose an efficient design point encoding, which describes both HW and mapping with a list of parameters. Our encoding method constructs a compact representation of the cross-coupled design space that boosts the efficiency of the optimization algorithms.
- We propose a domain-aware genetic algorithm-based optimization method, named DiGamma ($\mathcal{F}$). It is specifically designed for HW-Mapping design space and comes with specialized optimization operators to step through the design space in a structured manner, and its HW exploration strategy respects the interaction between HW and mapping.

We provide evaluations across two different settings: edge and cloud platform resources, and seven popular DNN models

with diverse characteristics. The evaluation shows DiGamma can achieve (geomean) **3.0x** and **10.0x** speedup, comparing to the best-performing baseline optimization algorithms, in edge and cloud settings, respectively, and (geomean) **1.25x** and **2.0x** speedup over the best-performing baseline HW optimization or Mapping optimization scheme, in edge and cloud settings.

II. BACKGROUND, MOTIVATION AND RELATED WORK

A. DNN Accelerators

DNN accelerator design points can often be described with two parts: HW resources and mapping strategy, described next.

1) **Hardware Resources**: Spatial DNN accelerators comprise an array of Processing Elements (PE). Each PE has a MAC to compute partial sums, and a local buffer (L1) to store weights, activations, and partial sums. The accelerators also house a shared global buffer (L2) to pre-fetch activations and weights from off-chip memory for the next tile of computation that will be mapped over the PEs and L1, as shown later in Fig. 3(d). Networks-on-Chip (NoCs) are used to distribute operands from L2 to L1 inside each PE and collect the partial or full outputs and write them back to the L2.

2) **Mapping**: A *mapping* [5] is comprised of: (1) tiling (how tensors are sliced, stored, and fetched across the memory hierarchy), (2) compute order (order in which loop computations are performed), (3) parallelism (how compute is mapped across PEs in space), and (4) clustering (how compute/buffer are structured into hierarchy of levels). In literature, (2)(3)(4) together are often called *dataflow* [1], [5].

B. Prior work in DNN Accelerator Design Space Exploration

HW Configuration Optimization. A typical HW resource optimizer [3] works as follows. It takes in a target DNN, the mapping strategy, an optimizing objective (e.g., latency, power) and resource constraints (e.g., area budget) as input. It returns a HW resource allocation (PEs and buffer sizes). The HW resource allocation problem has been widely-studied at *compile-time* in the FPGA community and at *design-time* for DNN ASIC accelerators. Some recent works apply ML techniques to the HW allocation problem such as reinforcement learning (RL) [3], GNN [6], and others (Fig. 1).

Mapping Optimizations. A typical mapping optimizer [4] takes in a target DNN, the *accelerator's HW configuration* (i.e., *resources*) as constraint, and an optimizing objective (e.g., latency, power). It can be used at *compile-time* or even at *run-time* (for reconfigurable accelerators). The mapping optimization process includes techniques such as formulating a more comprehensive design space for better capturing the accelerator behavior, improving the design space description for search efficiency, and incorporating different ML techniques in the optimization process [4], [11] (Fig. 1).

C. Challenges with HW-Mapping Co-optimization

A HW-Mapping co-optimizer can enable mapping-aware HW design and further optimize the accelerator design at *design-time*. However, designing a HW-Mapping co-optimizer is not trivial. The design space is the cross-product of HW space (as large as $O(10^{12})^1$) and mapping space (as large $O(10^{24})$ [4]), which

¹Assuming PEs:128x128 and maximum buffers:100MB, the number of combinations is $O(10^{12})$.

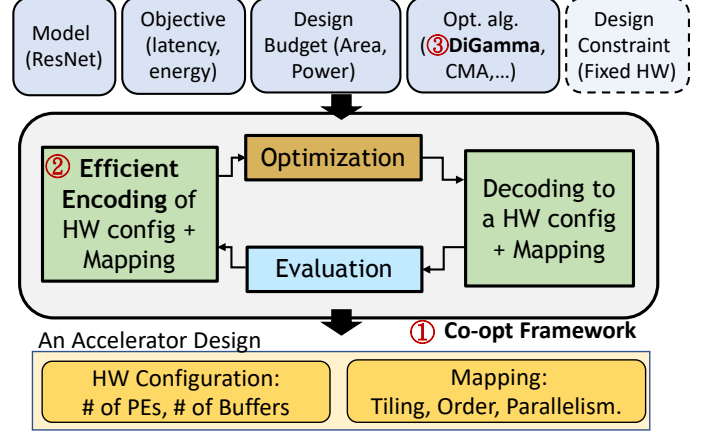


Fig. 2: HW/Mapping Co-optimization Framework. Our three technical contributions are highlighted.

can lead to a design space as large as $O(10^{36})$. Therefore basic techniques like exhaustive searches become impractical. An optimization-based algorithm (e.g., RLs, GA, simulated annealing, and so on) is needed. A naive optimization-based HW-Mapping co-optimizer can be formulated as follows. One can formulate a two-loop optimization process, where the outer-loop optimizes the HW, and the inner-loop (takes in the HW parameters from outer-loop) optimizes the mapping or vice versa. For e.g., a highly tuned mapper GAMMA [4] requires about 10 mins to converge to a mapping solution of a given HW configuration. For a two-loop optimization, the found solution at inner-loop (mapper) becomes the feedback of one single sampling point of HW optimizer at outer-loop, where outer-loop can easily require more than 10K sampling points. A naive two-loop optimization requires 1.6M sampling points and more than 1,600 hours, which is challenging for practical usage.

III. TECHNICAL APPROACH

Problem Formulation. Under a design budget (e.g., chip area) and given a DNN model, we aim to design an accelerator with optimized HW resource configuration for PEs, local buffer (L1), and global buffer (L2), and an optimized mapping strategy.

A. High-level Overview

We integrate the two searching loops (Sec. II-C), HW and mapping, into one unified optimization process, and propose ① a HW-Mapping co-optimization framework (Co-opt Framework), as shown in Fig. 2. Note that there are two main factors deciding the effectiveness of an optimization framework: efficiency of the design-point encoding and efficiency of the optimization/ search algorithm. In this work, we propose ② an efficient encoding method for HW configuration and mapping (Sec. III-C) and ③ a sample-efficient optimization algorithm (DiGamma) (Sec. IV).

B. HW-Mapping Co-optimization Framework

Co-opt Framework takes the input of target model, optimization objective, design budget, an optimization algorithm, and (optionally) a design constraint, and generates an optimized accelerator design point with HW configuration and mapping strategy. The design constraint is optional, for supporting two additional use-cases: 1) Fixed-HW: when the researcher/

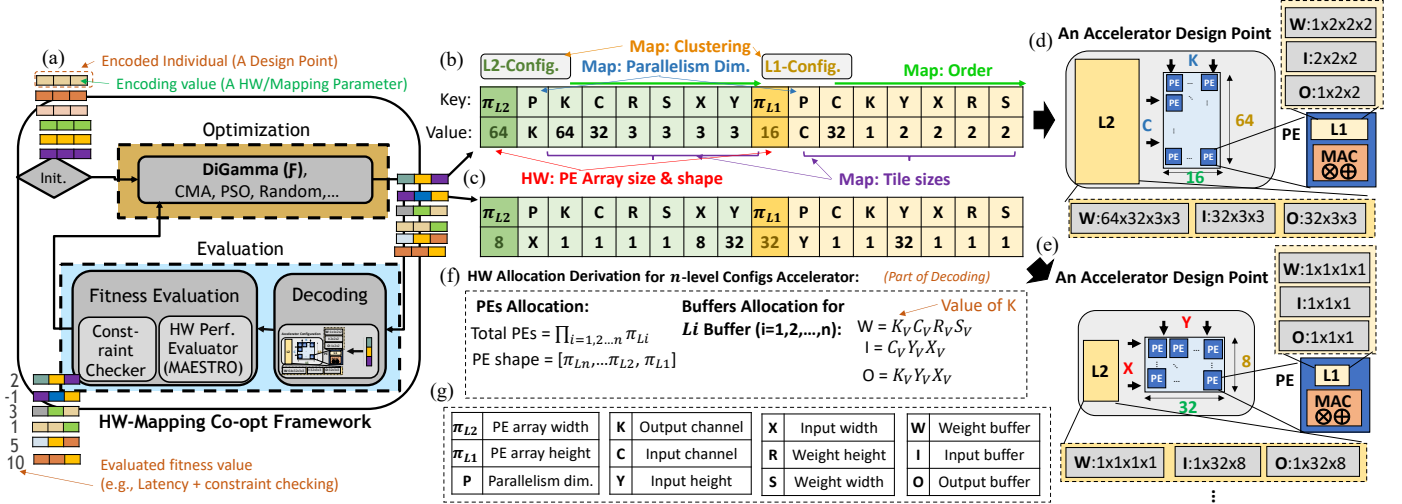


Fig. 3: (a) Co-opt Framework, (b-c) the HW-Mapping encoding representation, and (d-e) the corresponding decoded accelerator configuration. (f) The formula for calculating minimum on-chip buffer requirement. (g) The definition of notations.

engineer already has a designed accelerator and only wants to search for an optimal mapping; 2) Fixed-Mapping: when the researcher has a manual-tuned mapping (e.g., NVDLA [8]) and wants to understand the optimal HW configurations for designing a specialized accelerator (e.g., understanding the compute to memory balance). Co-opt Framework can deal with these constraints by restricting the design space accordingly. As shown in Fig. 3(a), Co-opt Framework includes an optimization block, where different optimization algorithms can be plugged and played, and an evaluation block, where the proposed design points are evaluated and scored to guide the algorithms.

1) **Optimization Block.**: One main goal of this work is to develop a generic framework for HW-Mapping co-optimization that is not tiled with any optimization/searching algorithms. We abstracted the underlying detail of taking different DNN models as inputs, understanding different design budgets, encoding/decoding of HW and mapping, and so on, and expose a generic interface for all the optimization algorithms. The only task left for any optimization algorithm (from any optimization library [9] or custom-designed) is to find a list of parameters (together forming a design point) that yields the highest reward (fitness). The sampling budget, which is the number of design points the algorithms are allowed to sample, is a hyper-parameter of optimization block that can be set by the users. It controls the number of optimization loops (optimization, evaluation, optimization,...) in the framework.

2) **Evaluation Block.**: The evaluation block includes a decoding module (described in Sec. III-C2) and a fitness evaluation module. The fitness evaluation includes a HW performance evaluator and a constraint checker (Fig. 3(a)).

HW Performance Evaluator. We evaluate all design points using an open-source HW performance evaluator, MAESTRO [5], which has detailed micro-architectural HW models and is validated against chip prototypes. MAESTRO takes in the accelerator design (HW and mapping) and outputs a detailed HW performance report including latency, area, power, energy, and so on. In the evaluation, we use HW resource area as the constrained design budget which includes the area of PEs, L1

and L2 buffers². **Constraint Checker.** In constraint checker, if the required resources (e.g., area or power) of the proposed design point is larger than the provided budget, we invalidate the design point by re-assigning it a negative fitness value.

C. Encoding of Design Point

1) **Encoding: Design Space Description.** One of our key contributions is a customized encoding, as shown in Fig. 3(b-c) (notation shown in Fig. 3(g)), to describe an accelerator design-point. Our encoding captures the compute resources, mapping, and the memory hierarchy. We show a 2-hierarchy level accelerator in Fig. 3(b-c) as an example. Each key-value pair represents a *gene*. L1-config (yellow) shows the accelerator configuration (HW and mapping) of a 1-D PE array. π_{L1} , a HW parameter, shows the length of the 1-D PE array. The rest of the genes (P, C, K, Y, X, R , and S) describe the mapping parameters for the 1-D PE array, including tiling, order, and parallelism. The value genes describe the tile sizes of the corresponding key dimension. The order of key genes describes the compute order. P gene tells the dimension to parallelize the compute across 1-D PE array. L2-config shows the HW and mapping across several 1-D PEs arrays, effectively describing a 2-D PE array. π_{L2} , a HW parameter, shows the number of instantiated 1-D PE arrays, while the rest mapping genes decide the mapping parameters across 1-D PE arrays. Similarly, a 3-level hierarchy (i.e., several 2D arrays) can also be described.

2) **Decoding: Design Point Derivation.** When evaluating the fitness/performance of each individual's genes, we decode them back to an exact accelerator design point. Fig. 3(d)(e) shows the decoded accelerator of the proposed design point by Fig. 3(b)(c), respectively. The π_{L2} and π_{L1} genes decide the PE array sizes and aspect ratio. Therefore different PE arrays are configured in Fig. 3(d)(e). The P values of L2 and L1 implies how the compute are fetched and parallelized to the PE arrays, e.g., K-C parallelism and X-Y parallelism in Fig. 3(d)(e). The order of the genes decides the computation order for each tile

²A complete chip area will also include NoCs, logics, routing, clock trees, pin placement, and others, which need other physical layout considerations and are not considered in MAESTRO and this paper.

in the PE arrays. Finally, tile sizes and levels of hierarchies (or called clustering) (e.g., two levels of cluster/ hierarchy in Fig. 3(b)(c)) determine the *minimum buffer requirement* to house weight, input, and output tensor at both L2 and L1 buffers.

IV. OPTIMIZATION ALGORITHM

A. Leveraging Existing Algorithms

With the thriving of AutoML, many optimization algorithms have been developed for automatically searching through a given design-space. They achieve state-of-the-art performance across many domains, including neural architecture search, AI-controlled game, chip design [6], and so on. **Co-opt Framework** provides a generic interface enabling us to plug and play many of these existing algorithms (which we leverage from nevergrad [9]), as shown in the experiments in Sec. V-B).

However, as discussed in Sec. II-C, the design space of HW-Mapping co-optimization is un-smoothed and extreme large. This challenges the search efficiency of the optimization algorithms and makes some of them ineffective under limited sampling budgets (Sec. V-B). This motivates our proposed algorithm, which customizes a genetic algorithm.

B. Background of Genetic Algorithm.

In genetic algorithm (GA), we often call an encoded value of a candidate a *gene*, each encoded candidate: an *individual*, a bag of candidates: a *population*, and one iteration of optimization loop: a *generation*. Baseline GA has two standard genetic operators: crossover (blend the genes of individuals and reproduce populations for the next generation) and mutation (perturb the genes of each individual).

Research shows GA reaches competitive performance with deep reinforcement learning [10], [12], and hyper-parameter optimization problem. Comparing to many optimizations methods, GA is light, fast, and highly parallelizable [10], [12]. However, the key challenge is its sample efficiency.

GAMMA [4] is an open-source genetic algorithm, tuned for DNN mapping optimization over given HW configurations of accelerators (Sec. II-B). Despite the effectiveness of GAMMA as a mapping search tool, using it naively to search for HW resources and mapping together will result in a two-loop optimization, which is extremely inefficient, as discussed in Sec. II-C.

C. DiGamma: Domain-aware Genetic Algorithm

DiGamma uses the encoding presented in Sec. III-C to describe design-points. It then perturbs these to search through the co-optimization space. Rather than using conventional genetic operators (crossover, mutations) to perturb the genes arbitrarily, which is shown to have poor sample efficiency (Sec. IV-B), we develop *specialized genetic operators* (i.e., optimization operators) for individual HW and mapping genes to capture the structure of the design space, as described next.

The genetic operators responsible for mapping (tiling, order, parallelism, clustering) are modified from GAMMA [4]. Additionally, we implement a *HW* genetic operator to perturb the PE configuration, where the values of π_{L2} and π_{L1} decide the total number of PEs and the aspect ratio of PE array. Further, for L1 and L2 buffer sizes, we employ a buffer allocation strategy

Operators	HW Space		Mapping Space			
	PE Array Shape & Size	Buffer Sizing	Tiling	Order	Parallelism	Clustering
Crossover	✓	✓	✓			
Reorder				✓		
Grow/Aging	✓	✓				✓
Mutate-Map		✓	✓		✓	
Mutate-HW	✓	✓				
✓: Adapted from Gamma. ✓: Added features in DiGamma.						

Fig. 4: DiGamma’s genetic operators[†] and their perturbing space[‡].
[†]: **Mutate-HW**: Mutation operating on HW space, which tweaks the PE size/shape and also affects the allocated buffer. **Mutate-Map**: Mutation operating on mapping space and co-affecting buffer choices in HW space.
[‡]: Definition of each space can be found in Fig. 1.

to decide the most optimized buffer allocations of a given individual (Fig. 3(b)(c)) by the *minimum buffer requirement* derived at decoding block (Sec. III-C2), i.e., we allocate the exact amount of buffer needed at both L2 and L1 to maximize buffer utilization. We summarize the developed specialized genetic operators and their different perturbing ability across HW and Mapping space in Fig. 4.

V. EVALUATIONS

A. Setup

Across our experiments, we use the optimizing objective of minimum latency which becomes the performance metric when evaluating the quality of the found solution. Other objectives can also be specified to DiGamma such as power, energy, EDP.

DNN Models. We experiment 7 DNN models across 3 popular DNN applications: vision (MobilenetV2, Resnet18, Resnet50, Mnasnet), language (BERT), and recommendation (DLRM, NCF).

Edge/ Cloud Platform Resources. We evaluate accelerator design under two types of platform resources: edge and cloud. We set the chip area budget for area of PEs and on-chip buffers as $0.2mm^2$ for accelerators in edge [1], [11] and $7.0mm^2$ for accelerators in cloud [11].

Area Cost Model. To estimate area cost, we implemented RTL of the various components in Fig. 3(d-e), synthesized them using Synopsys DC with Nangate 15nm library and used Cadence Innovus for place-and-route. We synthesized the SRAM buffers with SAED32 education library from Synopsys.

Sampling Budget of Optimization Algorithms. We set the sampling budget (maximum number of sampled points throughout the search process) as 40K points for all the investigated optimization algorithms. For DiGamma, it means population size times number of generations cannot exceed 40K, which takes about 20 mins of CPU-time.

Baseline Optimization Algorithms. We take 8 other optimization algorithms, which are widely-used and achieving state-of-the-art performance across different tasks, as baselines. The algorithms include: Random search, standard GA (stdGA), Particle Swarm Optimization (PSO), Test-based Population-Size Adaptation (TBPSA), (1 + 1)-evolution strategy ((1+1)-ES), Differential Evolution (DE), Passive Portfolio (Portfolio), and Covariance matrix adaptation evolution strategy (CMA).

Values	Latency									Latency-Area-Product								
Scheme	HW-Map-co-opt									HW-Map-co-opt								
Method	Random	stdGA	PSO	TBPSA	(1+1)-ES	DE	Portfolio	CMA	DiGamma	Random	stdGA	PSO	TBPSA	(1+1)-ES	DE	Portfolio	CMA	DiGamma
Platform Resources: Edge																		
Resnet18	N/A	4.7	169.3	N/A	N/A	0.8	0.8	1.0	0.5	N/A	6.3	46.2	N/A	N/A	1.2	1.2	1.0	0.7
Resnet50	N/A	N/A	110.9	0.7	21.7	1.1	0.3	1.0	0.3	N/A	N/A	12.0	0.7	20.8	1.1	0.3	1.0	0.3
Mbnet-V2	N/A	29.0	1.0	0.5	0.5	0.2	3.7	1.0	0.2	N/A	20.0	1.1	0.4	0.6	0.2	2.7	1.0	0.2
Mnasnet	N/A	5.3	27.6	0.1	33.5	0.04	0.04	1.0	0.04	N/A	1.0	2.5	0.1	35.3	0.04	0.04	1.0	0.05
BERT	N/A	N/A	N/A	1.0	2.0	10.0	0.5	1.0	0.4	N/A	N/A	N/A	0.7	2.6	13.2	0.7	1.0	0.5
NCF	229.8	88.4	N/A	133.6	0.5	N/A	1.9	1.0	0.4	179.3	78.4	N/A	105.0	0.5	N/A	1.9	1.0	0.4
DLRM	304.7	151.5	1.2	168.8	2.8	5.2	15.6	1.0	0.8	211.2	83.1	1.2	140.5	1.6	5.2	15.8	1.0	0.7
GeoMean	264.6	24.9	14.4	2.9	3.2	0.8	0.9	1.0	0.3	194.6	15.3	4.5	2.4	3.1	0.9	1.0	1.0	0.3
Platform Resources: Cloud																		
Resnet18	N/A	64.9	1652.5	0.4	373.6	N/A	0.3	1.0	0.03	N/A	30.3	19.1	0.1	201.7	N/A	0.3	1.0	0.03
Resnet50	N/A	116.2	1904.7	3593.7	N/A	1.5	6.4	1.0	0.3	N/A	113.9	262.8	1352.9	N/A	1.4	4.5	1.0	0.3
Mbnet-V2	0.8	0.8	20.8	0.8	8.9	3.2	0.6	1.0	0.4	1.4	1.4	28.6	1.4	20.5	6.1	1.3	1.0	0.7
Mnasnet	N/A	10.8	0.2	0.1	0.2	N/A	10.6	1.0	0.1	N/A	7.2	0.2	0.1	0.3	N/A	12.0	1.0	0.1
BERT	10669.5	18.9	27.2	13173.1	1043.3	42.3	2.3	1.0	1.2	1528.0	14.6	12.0	7659.3	990.3	35.8	2.4	1.0	1.3
NCF	47.1	15.0	0.6	0.3	41.2	0.6	0.4	1.0	0.01	39.2	10.8	0.8	0.4	14.9	0.8	0.6	1.0	0.01
DLRM	3481.5	17.1	0.8	28.0	54.1	4230.0	27.9	1.0	0.6	2098.1	8.8	0.6	16.0	53.0	760.0	26.1	1.0	0.4
GeoMean	193.3	16.4	14.5	10.8	34.6	13.8	2.3	1.0	0.1	115.7	12.6	5.5	7.3	30.6	11.4	2.5	1.0	0.1

Fig. 5: Performance comparisons of different optimization algorithms. Both latency and latency-area-product are normalized by the values of CMA, the best-performing baseline (lower is better). We highlight the best performing algorithm in different tasks (DNN models) in bold. N/A means the algorithm cannot find valid solution that fits in the area constraint under the set 40K sampling budgets.

Value	Latency					
Scheme	HW-opt (Optimizing HW, Fixed Mapping)			Mapping-opt (Fixed HW, Optimizing Mapping)		
Method	Grid-S HW + dla-like	Grid-S HW + shi-like	Grid-S HW + eye-like	Buffer- focused + Gamma	Medium- Buf-Com + Gamma	Compute- focused + Gamma
Platform Resources: Edge						
Resnet18	7.8	94.8	29.8	3.7	2.4	1.0
Resnet50	3.6	88.4	27.8	3.7	1.9	1.0
Mbnet-V2	2.4	48.5	15.9	2.9	1.8	1.0
Mnasnet	2.5	61.2	20.5	3.5	2.0	1.0
BERT	1.3	4.1	1.1	5.6	2.8	1.0
NCF	1.8	908.0	482.4	5.2	2.7	1.0
DLRM	1.8	890.8	473.4	4.7	2.4	1.0
GeoMean	2.6	97.4	35.3	4.1	2.3	1.0
Platform Resources: Cloud						
Resnet18	8.6	877.6	275.7	1.1	1.2	1.0
Resnet50	3.4	786.9	141.0	1.1	0.9	1.0
Mbnet-V2	3.2	276.4	90.6	1.1	0.9	1.0
Mnasnet	3.6	322.6	107.9	1.1	0.9	1.0
BERT	1.0	100.3	26.6	3.0	1.5	1.0
NCF	1.0	12288.1	6528.1	2.3	1.2	1.0
DLRM	1.3	10846.3	5763.9	1.8	1.1	1.0
GeoMean	2.4	972.6	324.8	1.5	1.1	1.0

Fig. 6: Latency of the found solution by different optimization scheme. Latency values are normalized by the values of best-performing baseline method (Compute-focused + Gamma). Grid-S: grid search. Buffer-focused: large buffer design. Compute-focused: large PE arrays design. Medium-Buf-Com: medium buffer and PE arrays design.

Baseline HW and Mapping Optimization Schemes. We formulate two kinds of HW and mapping optimization schemes and compare them with DiGamma, listed as follows.

- **HW-opt:** optimizing HW while mapping is fixed. The HW is optimized by grid search approach over number of PEs and buffer sizes. Note that the entire HW configuration design space is as large as $O(10^{12})$, which is hard to enumerate through, and therefore we use grid search. For mapping, we use the manual-designed NVDLA (dla)-like [8], ShiDianNao (shi)-like [2], and Eyeriss (eye)-like [1].
- **Mapping-opt:** optimizing mapping while HW is fixed. The mapping is optimized by GAMMA [4], a mapping optimizer for a given HW configuration. We cherry-picked three sets of HW configurations: Buffer-focused (small compute + large buffer), compute-focused (large compute + small buffer),

HW-opt (Grid-S HW + dla-like)	π_{L2}	P	K	C	Y	X	R	S	π_{L1}	P	K	C	Y	X	R	S
	7	K	7	64	3	3	3	3	64	C	1	64	3	3	3	3
Mapping-opt (Compute- focused HW + Gamma)	π_{L2}	P	S	R	X	K	Y	C	π_{L1}	P	C	X	R	S	Y	K
	56	K	3	1	9	56	1	3	9	X	1	9	1	3	1	1
HW-Map-co-opt (DiGamma)	π_{L2}	P	Y	C	R	K	X	S	π_{L1}	P	K	S	C	X	R	Y
	33	K	7	2	2	33	83	3	12	X	1	3	1	12	1	1

Scheme	Latency (cycles)	Area (mm ²)	Lat-Area- Product	PE Area : Buffer Area Ratio
HW-opt (dla-like)	3.74E+06	1.99E+05	7.44E+11	67 : 33
Mapping-opt (Com-focused)	1.14E+06	1.83E+05	2.08E+11	39 : 61
HW-Map-co-opt (DiGamma)	9.80E+05	1.99E+05	1.95E+11	56 : 44

Area Constraint: 2.00E+5 (mm²)

Fig. 7: The solution found by different optimization schemes and their corresponding performance on Mnasnet at edge resources.

and Medium-Buf-Com (medium Buffer + medium compute) configuration for both edge and cloud settings. Note that the design is area constrained, therefore compute and buffer resources are traded-off with each other.

- **HW-Map-co-opt:** using DiGamma to co-optimize both HW and mapping³.

B. Comparisons with Baseline Optimization Algorithms

Fig. 5 shows the achieved performance (latency) by different optimization algorithms. Note that the proposed Co-opt Framework is a supportive back-end and can work well with many state-of-the-art algorithms such as DE, Portfolio, and CMA. Considering both stability (without N/A) and performance, CMA is the best-performing one among the compared baseline algorithms. The performance value of CMA represents the best performance that Co-opt Framework could bring before discussing the new algorithm, DiGamma. Therefore, we normalize the values in Fig. 5 by the value of CMA. We walk through more detail of Fig. 5 and discuss DiGamma's performance, next.

At edge settings, some algorithms can achieve compatible performance with DiGamma in specific tasks such as DE and Portfolio in Mnasnet, however not stable with the respect to

³The hyper-parameters of DiGamma, (mutation rate, crossover rate, elite ratio, population size to number of generations ratio, and so on), are decided by a Bayesian optimization-based search process [7].

the stability across tasks (Fig. 5). E.g., DE did not find any solution in NCF, and Portfolio performed 15.6x worse than CMA in DLRM. Besides the latency value of different solutions, we also show their corresponding latency-area-product, since some solution/designs could trade-off areas for better latency. E.g., in BERT cases, TBPSA ranks 2nd across 8 baseline algorithms in latency performance. However, with the respect to latency-area-product, TBPSA is the best among baseline algorithms, meaning it has better area efficiency to achieve similar latency performance comparing to others. Moreover, the poor performance of standard GA contrasts the effectiveness of DiGamma's domain-aware optimization operators. **At cloud settings**, the wider design space in cloud cases increases the complexity of the optimization tasks. Multiple algorithms are not able to find any valid solutions or can only achieve relatively bad performance. In addition, some performance-leading algorithms such as DE, Portfolio, and CMA at edge settings, become unstable and have much larger swing of achieved performance across different tasks/models at cloud settings. In contrast, DiGamma can stably achieve compatible or better performance than others. Overall, comparing to best-performing baseline algorithm (CMA, Portfolio), DiGamma achieves (geomean) **3.0x** and **10.0x** better latency performance at edge and cloud settings, respectively.

C. Comparisons with Baseline HW-Mapping Schemes

In Fig. 6 we found that, among the compared methods, using heuristic HW configuration (Compute-focused) with existing mapping searching tool (GAMMA) can yield the best performance, whose value is thus used to normalized the values in Fig. 6. It represents the best achievable relative performance gain before proposing Co-opt Framework and DiGamma. We describe more detail of Fig. 6, next.

Comparing with HW-opt. In this experiment, we model the scenario that the researchers designed an optimized mapping for certain DNN models such as NVDLA [8], ShiDianNao [2], and Eyeriss [1] mapping and want to explore their performance across different models. However, different tasks/ DNN models expose different characteristics (e.g., in general, CNNs are more compute-intensive, and recommendation models are more memory-intensive). To achieve better performance, the researcher could apply optimization algorithms to search for optimal HW configurations (PEs and buffers). Here, we use the grid search approach to search through different PEs and buffers configurations. In Fig. 6, we could observe that DiGamma constantly achieve better performance across three different HW-opt methods, where DiGamma is (geomean) **3.25x** and **4.8x** better than the best-performing method (Grid-S + dla-like), in edge and cloud settings, respectively.

Comparing with Mapping-opt. In this experiment, we model the scenario that the researchers designed a mapping optimization algorithm, however relying on a pre-defined HW configuration, which become an inductive bias from the human. For example, different researchers will design different balances between compute and memory resources, where we model with three sets of configurations (Sec. V-A). Note that the previously effective strategy of exhaustive grid search does not fit this

scenario, since grid searching HW plus mapping optimizations will form two loops of the optimization process, whose required sampling budget and search time increase exponentially. In Fig. 6, we can observe that DiGamma is **1.25x** and **2.0x** better than the best-performing method (Compute-focused + Gamma), in edge and cloud settings, and more importantly, without the need of human-in-the-loop to cherry-pick the HW configurations.

D. Explanation of Found Solutions

Fig. 7 shows three solutions of different optimization schemes for Mnasnet at edge resources. In HW-opt, we could observe the output/input-channel (K-C) parallelism features of dla-like mapping. In Mapping-opt, we could observe the mapping optimizer find an unique mapping strategy, channel and activation (K-X) parallelism, which is different from dla-like (K-C), shi-like (Y-X), and eye-like (Y-R, row-stationary). DiGamma also finds a mapping with K-X parallelism for Mnasnet, however with better compute and buffer balance, therefore achieving 3.8x and 1.6x better performance than HW-opt and Map-opt.

VI. CONCLUSION AND TAKEAWAY

DNN accelerator design often requires extensive tuning for HW resources and mapping due to a large diversity in DNN models and huge design-spaces for HW and mapping. Recent works show the benefit of either optimizing HW configurations (HW-opt) or mapping configurations (Mapping-opt) independently can harvest several order performance gains compared to fixed HW and mapping. This work shows that co-optimizing both HW and mapping together can (1) yield another tens to hundreds of performance gain comparing to HW-opt and Mapping-opt, (2) more importantly, within the same sampling budget as previous schemes, and (3) largely reduce the repeated engineering cost by minimizing the human decisions in the optimization/design loop of new DNN accelerators for new DNN models, especially, in the era of fast-evolving DNNs.

REFERENCES

- [1] Y.-H. Chen *et al.*, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *ISCA*, 2016.
- [2] Z. Du *et al.*, "Shidiannao: Shifting vision processing closer to the sensor," in *ISCA*, 2015.
- [3] S.-C. Kao *et al.*, "Confucius: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning," in *MICRO*, 2020.
- [4] S.-C. Kao *et al.*, "Gamma: automating the hw mapping of dnn models on accelerators via genetic algorithm," in *ICCAD*, 2020.
- [5] H. Kwon *et al.*, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *MICRO*, 2019.
- [6] A. Mirhoseini *et al.*, "A graph placement methodology for fast chip design," *Nature*, 2021.
- [7] F. Nogueira, "Bayesian Optimization: Open source constrained global optimization tool for Python," 2014-. [Online]. Available: <https://github.com/fmfn/BayesianOptimization>
- [8] NVIDIA, "NVIDIA Deep Learning Accelerator (NVDLA)," <https://nvidia.org>, 2018.
- [9] J. Rapin *et al.*, "Nevergrad - A gradient-free optimization platform," <https://github.com/facebookresearch/nevergrad>, 2018.
- [10] T. Salimans *et al.*, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv:1703.03864*, 2017.
- [11] Y. S. Shao *et al.*, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *MICRO*, 2019.
- [12] F. P. Such *et al.*, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv:1712.06567*, 2017.